



Department of IT and Computer Science
Pak-Austria Fachhochschule: Institute of Applied Sciences & Technology

Subject: Artificial Neural Network
Complex Computing Project (CCP)

Class: BSAI F23 Red/Blue

Due Date: 25-04-2026 through MS Teams.

Weightage: 5%

Course Learning Outcomes:

By the end of this project, students will be able to:

- Understand the biological basis of neural computation and implement a Perceptron model for binary classification tasks.
- Apply the Delta Rule to train Adaline networks through gradient descent and analyze learning rate effects on convergence.
- Design and train Kohonen Self-Organizing Maps for unsupervised feature learning, clustering, and topology-preserving dimensionality reduction.
- Implement a multi-layer Backpropagation network from scratch to solve complex non-linear classification problems.
- Evaluate and compare neural network models using performance metrics including accuracy, confusion matrices, and loss curves.
- Integrate multiple neural learning paradigms into a unified system and critically analyze their strengths and limitations.

Overview

Task Description

What you need to do:

Project Overview:

This project focuses on building a comprehensive Neural Computing System capable of recognizing and classifying handwritten digit patterns (0–9) from the MNIST dataset. You will



implement four fundamental neural learning paradigms sequentially, each contributing a distinct role in the overall intelligent system pipeline. The project bridges classical neural computing theory with modern implementation practice.

Objective:

- Implement a single-layer Perceptron to establish a baseline binary classifier distinguishing digit classes.
- Train an Adaline unit using the Delta Rule (Widrow–Hoff learning) and analyze gradient-based weight convergence.
- Build and train a Kohonen Self-Organizing Map (SOM) to cluster digit feature representations in an unsupervised manner.
- Design and implement a multi-layer Backpropagation network for full 10-class digit classification.
- Evaluate each model with appropriate performance metrics and produce a comparative analysis of all four paradigms.

Steps to Follow:

Part A: Perception and the Perceptron Model

Implement the foundational Perceptron model based on the biological neuron analogy. The Perceptron receives weighted inputs, applies a threshold activation, and produces a binary decision. This part establishes the conceptual and mathematical groundwork for all subsequent models.

The implementation must include the following steps:

i. Biological Mapping:

Explain and diagram the correspondence between biological neuron components (dendrites, soma, axon) and the mathematical Perceptron (inputs, weights, activation function). Discuss the historical significance of the Perceptron and its limitations.

ii. Perceptron Implementation:



Implement a single-layer Perceptron in Python using NumPy only. Train it on a binary subset of MNIST (e.g., digits 0 vs. 1). Plot the decision boundary evolution over training epochs. Apply the Perceptron update rule: $w(t+1) = w(t) + \eta \cdot (d - y) \cdot x$

iii. Convergence Analysis:

Demonstrate Perceptron convergence on linearly separable data. Empirically show its failure on non-linearly separable problems and explain the theoretical reason. Document epoch-by-epoch weight changes and misclassification counts.

Part B: The Delta Rule and Adaptive Linear Neurons

Implement the Delta Rule (Widrow–Hoff learning) to train an Adaline (Adaptive Linear Neuron) unit. Unlike the Perceptron, Adaline uses a continuous linear output during training and minimizes mean squared error via gradient descent, forming the theoretical bridge to Backpropagation.

The implementation must include the following steps:

i. Mathematical Derivation:

Derive the Delta Rule update equation from the MSE loss function $E = \frac{1}{2} \sum (d - \hat{y})^2$ using gradient descent. Show all partial derivative steps. Present the update rule: $\Delta w_i = \eta \cdot (d - \hat{y}) \cdot x_i$

ii. Adaline Implementation:

Implement an Adaline unit from scratch using NumPy. Train on a real-valued regression or binary classification task using continuous-output gradient descent. Plot the MSE loss surface and the gradient descent trajectory over weight space.

iii. Learning Rate Analysis:

Experiment with learning rates $\eta \in \{0.0001, 0.001, 0.01, 0.1\}$. Produce convergence curves for each and discuss the trade-off between convergence speed, oscillation risk, and stability. Identify the optimal learning rate for your dataset.

Part C: Kohonen Self-Organizing Maps (SOM)

Build and train a Kohonen Self-Organizing Map on high-dimensional MNIST feature vectors. SOMs perform unsupervised competitive learning, mapping input distributions onto a 2D grid while preserving topological relationships. The trained SOM provides cluster labels used as auxiliary features in Part D.

The implementation must include the following steps:

i. Architecture and Competitive Learning:

Describe the SOM architecture: input layer, 2D competitive neuron grid, weight vectors, Best Matching Unit (BMU) selection, and the Gaussian neighborhood function. Implement BMU selection as: $\text{BMU} = \text{argmin}_i \|x - w_i\|$ for all neurons i .

ii. SOM Training:

Implement the full SOM training algorithm from scratch. Use a decaying learning rate and neighborhood radius: $\eta(t) = \eta_0 \cdot \exp(-t/\lambda)$ and $\sigma(t) = \sigma_0 \cdot \exp(-t/\lambda)$. Apply the weight update: $w_i(t+1) = w_i(t) + \eta(t) \cdot h(\text{BMU}, i, t) \cdot (x - w_i(t))$

iii. Visualization and Evaluation:

Train on a representative MNIST subset and visualize: (a) the U-matrix showing neuron distances, (b) weight maps for each neuron plotted as 28×28 digit images, (c) class hit maps showing which digit classes activate which neurons. Compute Quantization Error (QE) and Topographic Error (TE) as quality metrics.

iv. Anomaly Detection:

Use the quantization error per input to detect anomalous samples (corrupted or ambiguous digits). Set a threshold based on the error distribution and evaluate the anomaly detector using precision and recall.

Part D: Backpropagation Networks

Design and implement a fully-connected multi-layer Backpropagation network for 10-class digit classification. Backpropagation extends the Delta Rule to hidden layers using the



chain rule for gradient computation. This is the most complex and highest-weighted component of the project.

The implementation must include the following steps:

i. Theoretical Derivation:

Derive the Backpropagation equations for a 3-layer MLP. Compute output-layer delta: $\delta_k = (d_k - y_k) \cdot f'(\text{net}_k)$ and hidden-layer delta: $\delta_j = f'(\text{net}_j) \cdot \sum_k \delta_k \cdot w_{kj}$. Show the general weight update: $\Delta w_{ij} = \eta \cdot \delta_j \cdot o_i$

ii. MLP Implementation:

Implement a fully-connected MLP with architecture $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$ using NumPy only. Implement forward pass, loss computation (cross-entropy), and full backpropagation. Train on the MNIST training set and target a test accuracy of $\geq 95\%$. Track and plot training and validation loss/accuracy per epoch.

iii. Activation Function Comparison:

Compare Sigmoid, Tanh, and ReLU activation functions in terms of convergence speed, gradient vanishing behavior, and final accuracy. Plot activation curves and gradient magnitudes. Justify your final choice of activation function.

iv. Regularization:

Implement L2 weight regularization (weight decay) and Dropout. Compare training vs. validation loss curves with and without regularization. Quantify the reduction in overfitting.

v. SOM Integration:

Incorporate the SOM cluster assignments from Part C as auxiliary input features to the MLP. Measure whether topological pre-processing improves classification accuracy or convergence speed compared to the baseline MLP.

Dataset:



Department of IT and Computer Science

Pak-Austria Fachhochschule: Institute of Applied Sciences & Technology

Use the MNIST Handwritten Digits dataset as the primary data source throughout all project parts.

You may obtain it from the following sources:

Link 1: <http://yann.lecun.com/exdb/mnist/>

Link 2: <https://www.kaggle.com/datasets/hojjatk/mnist-dataset>

Link 3: Available directly via torchvision.datasets.MNIST or tensorflow.keras.datasets.mnist (for loading only — all model implementations must be from scratch using NumPy).

Instructions for Writing the Report

Instructions for Report

Length and Structure: Write a structured report of at least 3000 words summarizing each project part with clear section headings, mathematical derivations, experimental results, and critical analysis.

Formatting:

Font Family: Times New Roman.

Font Size: 12 pt.

Line Spacing: 1.5.

Alignment: Justified.

Cover Page: Include project title, your name, class, submission date, and instructor name.

Page Numbers: Add page numbers at the bottom center of each page.

Content:

- **Introduction:** Describe the project objectives, the MNIST dataset, and the neural computing paradigms to be explored.
- **Part A Report:** Include biological analogy diagram, Perceptron pseudocode, decision boundary plots, and convergence analysis.
- **Part B Report:** Include full Delta Rule derivation, Adaline loss surface plot, and learning rate comparison curves.



- Part C Report: Include SOM training algorithm, U-matrix visualization, class hit maps, QE/TE metrics, and anomaly detection results.
- Part D Report: Include full Backpropagation derivation, MLP training curves, activation function comparison, regularization analysis, and SOM integration results.
- Conclusion: Comparative analysis of all four neural paradigms in terms of computational complexity, representational power, and suitability for pattern recognition.

Captions: Provide clear captions for all figures, plots, tables, and code listings.

Submission: Compile the complete project as a ZIP archive containing the PDF report, all Python source files, and Jupyter notebooks. Submit via MS Teams before the due date.

Rubric for Complex Computing Project

Criteria	Exemplary (90–100%)	Proficient (75–89%)	Adequate (50–74%)	Needs Improvement (<50%)	Weight-age
Perceptron Implementation & Theory	Correct implementation with full biological mapping, convergence proof, and insightful XOR failure analysis.	Working Perceptron with biological analogy and convergence shown; limited failure analysis.	Partial implementation; convergence not clearly demonstrated.	Perceptron missing or non-functional; no theoretical explanation.	20%
Delta Rule Derivation & Adaline	Complete gradient derivation with correct Adaline implementation; thorough learning rate study with justified conclusions.	Derivation provided; Adaline works; limited learning rate analysis.	Partial derivation; Adaline partially functional; minimal analysis.	No derivation; Adaline missing or incorrect.	20%
Kohonen SOM Training & Visualization	Well-trained SOM with U-matrix, weight maps, class hit maps; QE/TE computed; anomaly	SOM trained; at least two visualizations provided; basic quality metrics.	SOM partially trained; limited or unclear visualizations.	SOM missing or untrained; no visualizations.	25%



Criteria	Exemplary (90–100%)	Proficient (75–89%)	Adequate (50–74%)	Needs Improvement (<50%)	Weight-age
	detection evaluated.				
Backpropagation Network & Integration	Full chain-rule derivation; MLP achieves $\geq 95\%$ accuracy; activation comparison done; regularization evaluated; SOM integration measured.	Derivation provided; MLP functional with good accuracy; some analysis of activation/regularization.	Partial derivation; MLP trains but accuracy below 90%; limited analysis.	Backpropagation not derived; MLP missing or non-functional.	35%
Report Quality & Comparative Analysis	Clear, well-structured, professional report with correct formatting, all figures captioned, and insightful comparative conclusion.	Report is clear and mostly complete; minor formatting or captioning issues.	Report present but lacks depth, figures, or proper formatting.	Poorly written, incomplete, or missing report.	Included above